Conference session

From Platform Engineer to Inference Engineer

LLMDAY NYC

Annie Talvasto · Upbound

The promise

Most of what you know still applies. A few things really are new.

Still yours

Scheduling, scaling, routing, observability and resource management.

Genuinely new

KV (Key-Value) caches, tensor parallelism, continuous batching, and prefill versus decode. Vocabulary first, then operations.

By the end

A mental model and enough vocabulary to operate an LLM inference service with confidence.

Why this matters now

Three requests end in the same place: run the model ourselves

The code can’t leave

The coding assistant can’t send source code to a third party.

The bills keep climbing

Frontier model spend is growing into a real line item on the budget.

The data stays home

The support tool needs internal data that can’t leave the network.

Open models are ready for this. Hugging Face hosts more than two million. The price of a fixed level of AI capability falls roughly 10x a year, mostly from how models are served, not from better GPUs.

Why this matters now

Your skills, in inference form

What you already know	What it becomes in inference
Scheduling and node pools	GPU placement with device selectors, taints and exclusive devices
Capacity planning	GPU memory math: weights, KV cache and runtime overhead, checked at runtime
Load balancing	Cache-aware routing and long-lived streaming connections
Autoscaling on CPU	Scaling on queue depth and KV cache use, with cold starts measured in minutes
Rollouts	Rollouts of exclusive GPUs, plus a version matrix of model, hardware and engine build

The role

An inference engineer is SRE for a new workload

01

Picks the model

02

Gets it onto the right GPU

03

Keeps it serving

04

Makes it faster and cheaper

05

Gets paged when it breaks

That’s SRE for a new workload. Most of your skills carry over, in a new shape.

The engine is one of four layers

Gateway	Routes traffic across models and replicas
Control plane	Deploys and manages engines. No request passes through it.
Engine	vLLM or SGLang, which turns model weights into an API
Kernels	Do the arithmetic on the GPU. Benchmarked often, rarely chosen.

The closest mental model: a stateful database server with a GPU attached.

Weights are the fixed cost. The KV cache is the working set.

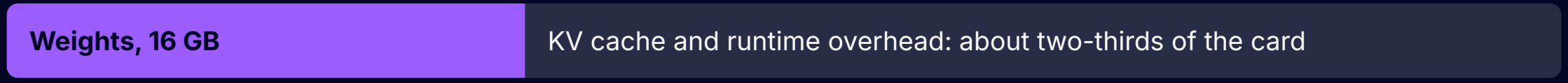
Weights: fixed

An 8B model at 2 bytes per parameter needs about 16 GB in GPU memory. That number doesn't move, whether one person or a hundred use it.

KV cache: grows with load

It scales with context length times concurrent conversations. Llama 3 70B at FP16 is about 320 KB per token, so one 128K-token conversation needs about 40 GB.

One 48 GB L40S, with an 8B model at FP16




```
KV bytes per token = 2 × layers × KV heads × head dimension × bytes per value
```

Inference 101

Every request runs two phases with different bottlenecks

Prefill

Reads the whole input in parallel: system prompt, tool definitions, conversation history. Produces the first token and the KV cache.

Limited by: compute

Sets: time to first token

Decode

Writes the answer one token at a time. Each token reads the active weights and the KV cache from memory, and at long context the cache can cost more than the weights.

Limited by: memory bandwidth

Sets: time per output token

A technique that speeds one phase often does little for the other. That’s why benchmarks report them separately.

Source: ByteByteGo; Data Gravity

Measure what users feel, and price it per million tokens

Time to first token

Queueing plus prefill. Measure the first token the user sees.

Time per output token

Decode speed per stream, where memory bandwidth shows up.

Throughput

Total tokens per second across all users. Rises with batch size.

Goodput

Throughput that still meets your latency targets. Plan capacity with this.

Cost per million tokens

```
GPU cost per hour ÷ (tokens per second × 3,600) × 1,000,000
```

$0.67

A GPU at $2.90 an hour sustaining 1,200 tokens per second.

Running it in production

Start from the workload, not the GPU

Coding agents

- Long context without quality loss
- Reliable tool-call parsing and streaming
- Sessions that last hours
- An Anthropic-compatible API for Claude Code

Chat

- Low time to first token
- Real concurrency
- An OpenAI-compatible API for existing tools

The platform

- TLS on a real hostname
- Identity tied to the company domain
- Observability that tells healthy from busy
- Correct placement without babysitting

Write the pass and fail bar down before you test anything. Separate interactive workloads from batch.

Running it in production

Size for runtime, not startup

A model that fits at startup can still run out of memory days later, under real traffic. Budget three costs: weights, the KV cache, and runtime overhead such as CUDA graphs, activations and fragmentation.

Cap every request

Set a maximum output length on every call, and cap concurrent sequences. Count context-length rejections as errors.

9

out-of-memory crashes in four days, from a chat model that fit cleanly at startup

4.6 GiB

freed by a lower memory fraction, expandable segments, and 16 concurrent sequences

Source: Upbound, Project Champagne, Part 2

Running it in production

Don’t round-robin inference requests

Route for the cache

One replica may already hold a conversation’s prefix. Use cache-aware routing, such as the endpoint picker in the Gateway API Inference Extension.

Route by model name

One hostname can serve many models, routed on the model field of each request.

Speak both APIs

Most tools expect the OpenAI API. Claude Code expects the Anthropic Messages API, which vLLM serves natively, including streaming and tool calls.

Running it in production

Timeouts stack up at every hop

Client → Gateway route → Load balancer → Engine

15 s

A route default that bounded the whole response

60 s

A load balancer idle timeout that cut off answers still streaming

1 min+

A reasoning model thinking before its first visible token

Raise every timeout on the path. A cut stream can still return HTTP 200 with valid tokens, and simply stop.

Source: Upbound, Project Champagne, Parts 2 and 3

Running it in production

Agents read their credential once

People

Browsers can’t attach a bearer token to page loads or preflight requests. Handle browser login with OIDC at the gateway.

Agents

A 60-minute identity token kills a session that runs for hours. Give agents long-lived keys or a credential that refreshes.

Probe what should be refused

No token

A garbage token

A key you didn’t sign

An expired key, or a revoked one

A forged caller header

Keep open: the health check

Running it in production

“Slow” has three different shapes

Queued

Running is at the limit, waiting is above zero, and the KV cache is full.

Add capacity or replicas.

Thinking

Time to first token looks fine, but each request writes many hidden tokens.

Look at output tokens per request.

Genuinely slow

Time per output token is up.

Look at the engine, the batch size and the hardware.

Measure the first token the user sees, not the engine’s first token. Reasoning models can hide thinking in front of it.

Source: Upbound, Project Champagne, Part 4

The costliest failures return no errors.

A truncated stream returned HTTP 200 with valid tokens.

A metrics API reported available while holding no data.

Monitoring sat pending for six days.

Make the silent failures loud.

Scale on inference signals, not CPU

Signals that matter

Queue depth

KV cache use

Requests in flight

CPU usage tells you almost nothing here.

Respect the cold-start wall

A new GPU node has to boot, pull a large engine image and load the weights. For a large model, loading weights alone can take half an hour.

Cache weights on shared storage in each cluster, so new replicas don’t download them again.

Scale-to-zero is harder than it sounds. By the time the node is ready, the spike may be over.

Promote every serving change through five tiers

Offline

Validate the config and run the rig's own checks. Minutes, no GPU.

Rehearsal

Run the full test against a fake gateway or a small model, so rig bugs don't burn a GPU window.

Smoke

A few real requests on target hardware: a long prompt, a tool call, a stream, and a refusal.

Load with evals

Sweep concurrency up to your bar, with the eval suite running alongside.

Canary

A small share of real traffic. Write the rollback trigger down first.

Exclusive GPUs need a Recreate rollout, or a rolling update deadlocks. Keep the old version ready until signals hold through a peak.

Running it in production

Agents change the shape of traffic

59K
Median prompt length, in tokens

125K
P90 prompt length, in tokens

2/3
Of prompt tokens hit the prefix cache, because agents resend the transcript

1 min
Of thinking before the first visible token from a reasoning model

About ten engineers ran Claude Code and other coding agents against a self-hosted model. If your platform works for agents, it will work for most other clients.

Source: Upbound, Project Champagne, Part 5

Running it in production

When a score is low, check three places before the model

1. The harness

Run your current hosted model first. If it scores below a weaker model, the harness is broken.

2. The infrastructure

Did the model get to answer? Timeouts, expired credentials and routing errors look like model failures. Mark those runs “not measured.”

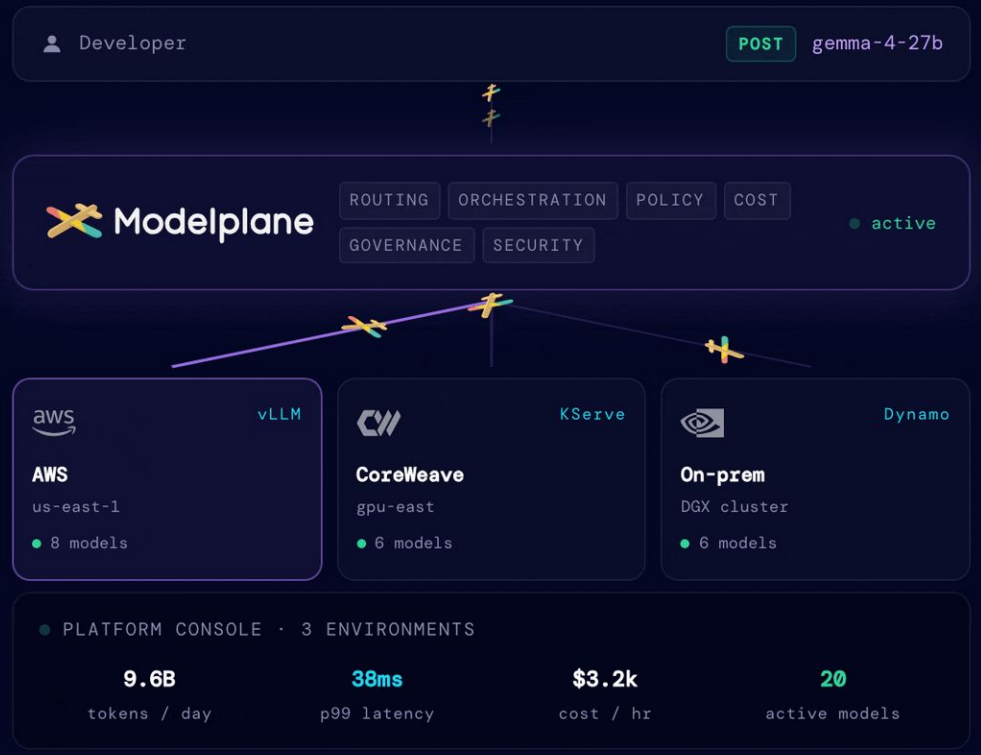
3. The grader

Grade the end state, not the agent’s description of it. Test graders against answers people already accepted or rejected.

Prefer deterministic checks over LLM judges, and run each test more than once. A batching server can return different tokens for the same request.

INTRODUCING MODELPLANE

The open source control plane for AI models. Built on Crossplane. Fleet-wide.



Modelplane

- Built on Crossplane • Specialized control plane
- Runs across cloud, neocloud, and data center
- Any model, any engine, any infra
- Open Source • Apache 2.0
- Launching in May into Crossplane's distribution
- Design partners in progress
- OS foundation donation planned

★ Star the Project

Star the Project

The stack

Modelplane splits ownership into three layers

Platform team

The fleet: clusters, hardware classes and the gateway.

`InferenceCluster`
`InferenceClass`
`InferenceGateway`

Model operator

The models: deployments, services and weight caches.

`ModelDeployment`
`ModelService`
`ModelCache`

Modelplane

Everything in between, composed from the two layers above.

`ModelEndpoint`
`ModelReplica`

The hierarchy mirrors Kubernetes, one scope up, across a fleet instead of one cluster:

```
ModelDeployment → ModelReplica → ModelService → ModelEndpoint

Deployment → Pod → Service → Endpoint
```

Source: Modelplane documentation, docs.modelplane.ai

1 · Fleet The hardware on offer

```
$ kubectl get inferenceclass -o custom-columns-file=scripts/columns/inferenceclass.txt
```

The tools, by layer

Layer	Tools to know
GPUs on Kubernetes	NVIDIA GPU Operator, Dynamic Resource Allocation, DCGM exporter
Inference engines	vLLM, SGLang, TensorRT-LLM; Ollama and llama.cpp for local work
Distributed serving	NVIDIA Dynamo, llm-d, KServe
Gateway and routing	Envoy Gateway, Agent Router, Gateway API Inference Extension
Control plane	Modelplane, built on Crossplane
Observability	Prometheus, Grafana, OpenTelemetry, Loki, Sloth or Pyrra
Benchmarking and evals	vLLM benchmark tools, AIPerf, GuideLLM, Inspect, Hugging Face upskill

Source: Platform Engineer to Inference Engineer guide, section 4

Case study: Project Champagne

Serving was the easy part

Quota isn’t capacity

Approved quota for 768 vCPUs of H100 instances never produced one. The planned GPU arrived about half the time.

“It fits” was a startup claim

A chat model that fit cleanly ran out of memory nine times in four days.

Timeouts stacked up

A 15-second route default, then a 60-second load balancer idle timeout, cut off answers that were still streaming.

The engine mattered most

On the same hardware, moving from vLLM to SGLang was the largest single gain measured.

Source: Upbound, Drinking Our Own Champagne series, 2026

Build it

Seven steps, from one GPU to a fleet

01 Run vLLM on one GPU behind its OpenAI-compatible API, and read its metrics endpoint.

02 Deploy it on Kubernetes with the NVIDIA GPU Operator, and place it with device selectors.

03 Put a gateway in front of it, with TLS, auth, and timeouts sized for long streams.

04 Scrape engine and GPU metrics into Prometheus, and build the core panels.

05 Benchmark with a concurrency sweep, while a small eval suite runs alongside.

06 Serve the same model on SGLang, and compare on your own traffic.

07 Split a larger model across GPUs with tensor and pipeline parallelism.

Career

Pick the direction closest to your work

Inference platform

Clusters, GPU scheduling, gateways, autoscaling and multi-cloud capacity.

Good fit if you enjoy Kubernetes and distributed systems.

Inference reliability

SLOs, observability, capacity planning, rollouts and incident response.

Good fit if you've carried a pager and run SRE practices.

Runtime performance

Engine tuning, quantization, speculative decoding, parallelism and kernels.

Good fit if you like profiling and working close to the hardware.

Most platform engineers start with the first two, then grow toward the third.

Career

Show your work in public

Write READMEs like lab reports

The question, the setup with engine version and flags, the method, results with numbers, what surprised you, and how to reproduce it.

Publish what failed

A clear account of a failure shows more skill than a clean demo.

Pin versions and show cost

Results without an engine version and a price are hard to trust, and harder to reuse.

Takeaway

Tell one failure story with real numbers: what broke, how you found it, and what you changed.

Takeaways

Remember these

1. Keep your platform skills, and add the inference concepts.
2. Measure per request, at the front door.
3. Make the silent failures loud.

Sources & where to go next

Upbound, Drinking Our Own Champagne series (Parts 1 to 5)

Philip Kiely, Inference Engineering (free e-book)

Viktor Farcic, LLM Inference Explained

Platform Engineer to Inference Engineer: docs.modelplane.ai

Next steps

Go deeper into inference

Inference Day

Hear where AI infrastructure is going from the engineers building it.

November 5, 2026 · 8 a.m. PST / 5 p.m. CET
- Kelsey Hightower fireside chat
- Open source maintainer panels
- Inference 101 with Viktor Farcic

Free and virtual. Save your spot.

Register Today

HOSTED BY Modelplane | SPONSORED BY upbound

New Today

Platform Engineer to Inference Engineer

Read the open field guide and dive deep into inference.

Read the guide

Open Source

Star Modelplane on GitHub

The open source control plane for AI inference. A star helps more platform engineers find it.

Star the Project